Mediator Pattern

Category: Behavioural Design Pattern

1. The Problem

Consider a chat system with multiple participants where each user can send messages to others. If users message each other directly, every user needs a reference to every other user.

```
class User {

    private String name;

    public User(String name){
        this.name = name;
    }

    public String getName() {
        return name;
    }

    public void sendMessage(String msg, User recipient){
        System.out.println(this.name + " sending " + msg + " to " + recipient.getName());
    }
}

public class WithoutMediatorProblem {
    public static void main(String[] args) {
        User sahil = new User("Sahil");
        User amit = new User("Amit");
        User aalia = new User("Aalia");
        User rahul = new User("Rahul");

        rahul.sendMessage("Hello", amit);
        rahul.sendMessage("Hello", sahil);
        rahul.sendMessage("Hello", aalia);
    }
}
```

Issues: - **High coupling** — as more users are added, each user needs to manage direct communication with every other user, creating a complex web of dependencies. - **Poor extensibility** — if a new communication rule is introduced (e.g. message logging), it would need to be added into every user's code individually.

2. The Solution — Mediator Pattern

Intent: Introduce a mediator object that handles all communication between objects, so those objects no longer need to know about — or directly depend on — each other.

- In the chat app, a `ChatMediator` (the `ChatRoom`) sits between users.
- Each `User` only ever talks to the mediator; the mediator is responsible for distributing messages to everyone else.

3. Key Idea

- **Centralize communication logic** — instead of every object holding references to every other object it needs to talk to, all objects hold a reference to a single mediator, and the mediator owns the logic for how they interact.
- **Program to an interface, not an implementation** — colleagues (`User`) depend only on the `ChatMediator` interface, not on a concrete list of other users.
- This converts a many-to-many web of dependencies into a simpler hub-and-spoke structure, where the mediator is the only object that needs to know about everyone.

4. Variants

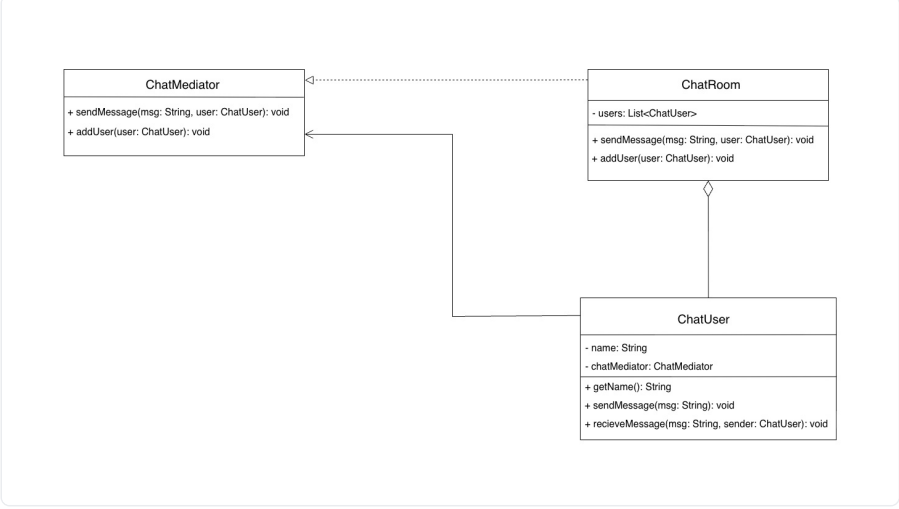
Not really applicable here — this exercise shows the standard Mediator structure. In larger systems, mediators are sometimes split into more specialized mediators for different concerns (e.g. a separate mediator for private messages vs. broadcast messages), but the core pattern remains the same.

5. Structure / Participants

Role	Responsibility	Example (from this exercise)
Mediator (interface)	Declares communication operations all mediators must implement	`ChatMediator` → `sendMessage()` , `addUser()`
Concrete Mediator	Implements the communication/coordination logic, holds references to colleagues	`ChatRoom`
Colleague	Communicates only through the mediator, never directly with other colleagues	`User`

Relationship type: Composition — each `User` *holds* a reference to a `ChatMediator` , and the `ChatRoom` (mediator) *holds* the list of `User` colleagues it coordinates.

Diagram



Mediator Pattern UML diagram

`ChatMediator` is the interface declaring `sendMessage()` and `addUser()`, implemented by `ChatRoom`, which holds a `List<ChatUser>` (composition, shown by the diamond). `ChatUser` holds a reference back to `ChatMediator` and exposes `sendMessage()` (delegates to the mediator) and `receiveMessage()` (called by the mediator to deliver an incoming message).

6. Code — Full Implementation

```
        import java.util.ArrayList;
        import java.util.List;

        interface ChatMediator {
            void sendMessage(String msg, User user);
            void addUser(User user);
        }

        class ChatRoom implements ChatMediator {
            private List<User> users;
            public ChatRoom(){
                this.users = new ArrayList<>();
            }
            @Override
            public void sendMessage(String msg, User sender) {
                for (User user : users){
                    if (user != sender) {
                        user.receiverMessage(msg, sender);
                    }
                }
            }
            @Override
            public void addUser(User user) {
                users.add(user);
            }
        }

        class User {
            private String name;
            private ChatMediator chatMediator;
            public User(String name, ChatMediator chatMediator){
                this.name = name;
                this.chatMediator = chatMediator;
            }
            public String getName() {
                return name;
            }
            public void sendMessage(String msg){
                chatMediator.sendMessage(msg, this);
            }
            public void receiverMessage(String msg, User sender){
                System.out.println(this.name + " received message: '" + msg + "' from " +
sender.getName());
            }
        }

        public class MediatorPattern {
            public static void main(String[] args) {
                ChatMediator mediator = new ChatRoom();
                User sahil = new User("Sahil", mediator);
                User amit = new User("Amit", mediator);
```

```
                User aalia = new User("Aalia", mediator);
                User rahul = new User("Rahul", mediator);

                mediator.addUser(sahil);
                mediator.addUser(amit);
                mediator.addUser(aalia);
                mediator.addUser(rahul);

                amit.sendMessage("Hello saablog");
            }
        }
```

What changed vs. the "without pattern" version: - Direct `User.sendMessage(msg, recipient)` calls → `User.sendMessage(msg)` now calls `chatMediator.sendMessage(msg, this)`. - No shared coordination point → `ChatRoom` (mediator) now holds the full user list and owns the broadcast logic. - Users had no relationship beyond direct calls → users now only hold a reference to the mediator, unaware of who else is in the chat. - Manual per-recipient looping → handled once, centrally, inside `ChatRoom.sendMessage()`.

7. Benefits

- **Reduces Complexity** — the mediator centralizes communication, reducing direct dependencies between objects.
- **Loose Coupling** — colleagues only interact with the mediator, making them easier to manage, extend, and maintain.
- **Single Responsibility** — the mediator handles complex communication logic, letting colleagues focus on their own behavior.
- **Centralized Control** — changes to communication rules (e.g. adding logging) can be made in the mediator without touching the colleagues.

8. Drawbacks / Things to Watch Out For

- The mediator itself can become a **"God object"** — as more coordination logic is added, it risks growing into a large, complex class that's hard to maintain.
- Since all communication flows through one place, the mediator can become a **single point of failure or bottleneck** if not designed carefully.

9. Real-World Use Cases

- **Air Traffic Control** — airplanes communicate through a central control tower (mediator) instead of coordinating directly with each other.
- **GUI Component Coordination** — in GUI apps, a mediator can handle interactions between components (e.g. a dropdown change triggering updates to text fields, buttons) instead of components knowing about each other directly.
- **Workflow Systems** — in business process management, a mediator can coordinate activities across multiple systems or departments.

10. Interview Questions & Answers

Q1. What is the Mediator Pattern? A behavioural design pattern that introduces a mediator object to handle communication between a group of objects (colleagues), so those objects no longer need direct references to each other.

Q2. What problem does it solve? It removes the tight coupling and complexity that builds up when objects communicate directly with many other objects — a change in communication rules would otherwise need to be replicated across every object involved.

Q3. What are its main components/participants? - **Mediator (interface)** — declares communication operations (`ChatMediator`) - **Concrete Mediator** — implements the coordination logic and holds references to colleagues (`ChatRoom`) - **Colleague** — communicates only through the mediator (`User`)

Q4. How does the Mediator Pattern change the shape of an object graph? It converts a many-to-many web of direct dependencies (every object knowing about every other object) into a hub-and-spoke structure, where every object only knows about the mediator, and the mediator is the only object that knows about everyone.

Q5. How does the Mediator Pattern differ from the Observer Pattern? Both decouple objects from each other, but their intent differs: **Observer** is about broadcasting state-change notifications from one subject to many interested observers (a one-to-many relationship). **Mediator** is about coordinating interactions *between* a group of peer objects (colleagues), often supporting more complex, two-way, many-to-many coordination logic — as seen here, `ChatRoom` could just as easily contain custom rules about who talks to whom.

Q6. What are the benefits? Reduced complexity, loose coupling between colleagues, single responsibility (coordination logic isolated in one place), and centralized control over communication rules.

Q7. What are the drawbacks? The mediator can turn into an overly large “God object” as more logic is added to it, and it can become a bottleneck or single point of failure since all communication passes through it.

Q8. Can you give real-world examples? Air traffic control towers coordinating planes, GUI frameworks coordinating interactions between components, and workflow/business process systems coordinating activities across departments.